Introductory Programming Techniques

North Metropolitan TAFE

ICTPRG302: Session 2

Courses:

- Certificate III in Information Technology
- Certificate IV in IT (Programming)
- Certificate IV in IT (Networking)
- Certificate IV in IT (Cybersecurity)
- Certificate IV in IT (Web)

Background created by Adrian Goold

What will I learn?

- The definition of a variable and an expression
- how to create and use variables to store data
- perform calculations using expressions.

To complete this week

- Weekly session exercises
- Portfolio questions
- Revision…practise, practise, practise!

Revision: Print Statements (function)

What can you remember?

- print(‘apple’)
- print(“apple”)
- print(“apple”, “ banana”)
- print("apple” + "banana")
- print(“1” + “2”)
- print(1+2)

Revision: Comments

- Anything typed after a hashtag # is ignored by Python
- Not shown on output screen, only visible in code view
- Describes what happens in a sequence of code
- Documents the author of the code
- Contains other information: Date, program name, notes etc
- Turns off a line of code – good for debugging.

Reminder: Additional resources

Freely available online courses, that can be useful to enhance your knowledge.

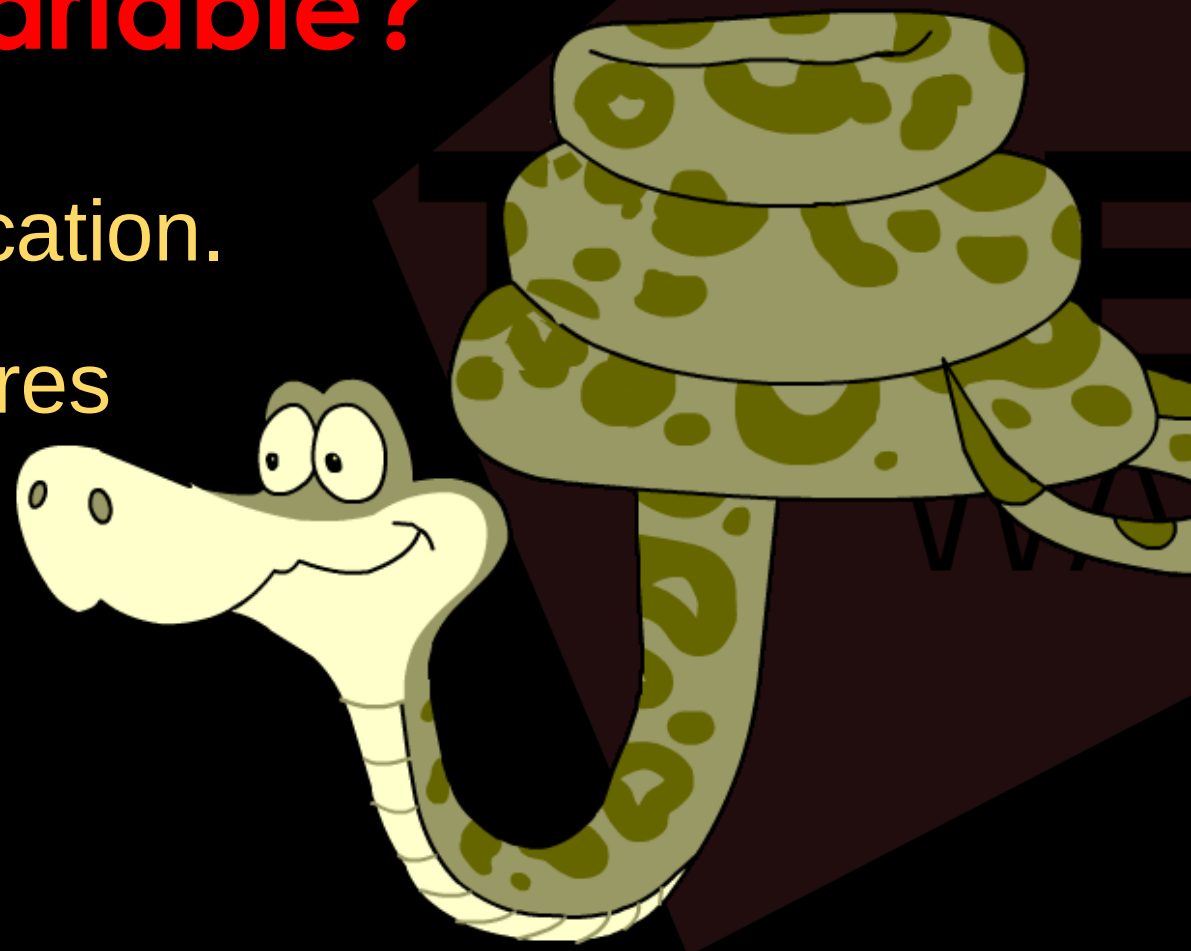
- Python for Everyone (Py4e)
- LearnPython.org
- W3schools

What is a Variable?

A variable is a named storage location.

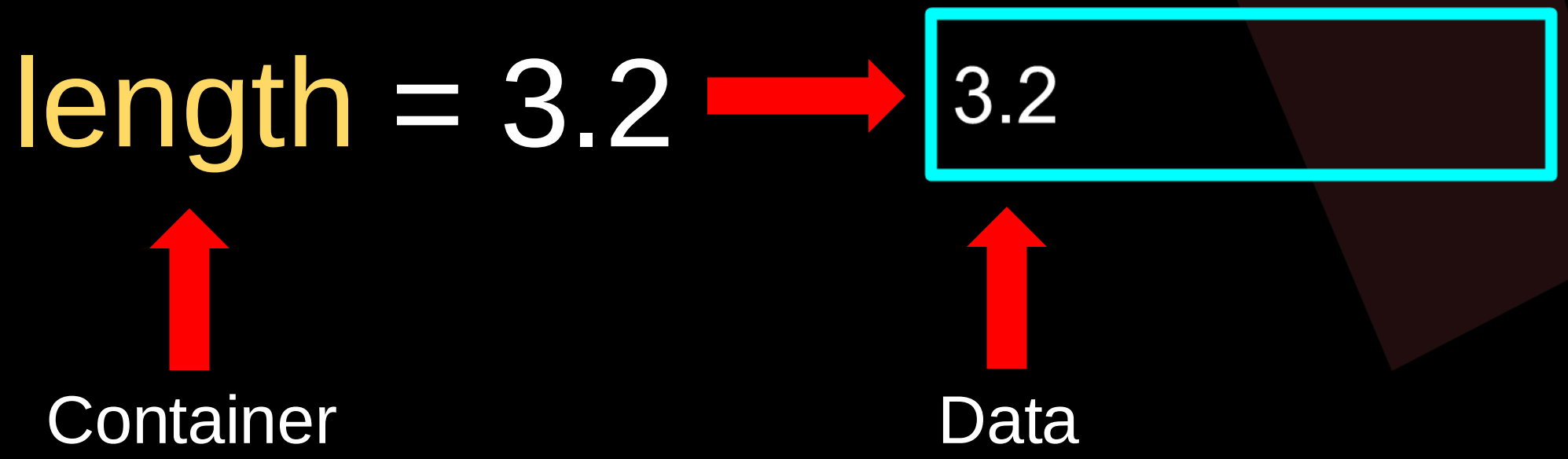
Think of it as a container that stores data such as:

- numbers
- Strings
- Other data types (explained in future sessions).



Variables

A variable is created when you assign a value (data) to it.



Variables

You can change the contents of a variable.

length = 3.2 → ~~3.2~~ 3.5

Changeable data

Rules: Python Variable Names

- Must start with a letter (abcd...) OR underscore (_)
- Must consist of letters, numbers and underscores
- Case sensitive

PEP8 standards:

Use lowercase, English words, separated by underscores.

Rules: Python Variable Names

Good: name hours hours24 _speed

Bad: 24spam #sign var.24

Not great: spam Spam SPAM

What does this bit of code do?

```
x1q3z9ocd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ocd * x1q3z9afd
print(x1q3p9afd)
```

What does this bit of code do?

```
a = 35.0
b = 12.50
c = a * b
print(c)
```

What does this bit of code do?

```
hours = 35.0
rate = 12.50
pay = hours * rate
print(pay)
```

Mnemonic Variable Names

Mnemonic: A technique used to help remember information.

Involves using patterns of letters, ideas, or associations to make the information easier to recall.

We name variables to help us remember what we intend to store in them ("mnemonic" = "memory aid").

Variable Names

- Should relate to the data being stored
- Must clearly describe the variable's purpose
  - For example, total_price is more descriptive than tp
- Choose short, meaningful, descriptive English words
- Avoid reserved words
- It is not a suitable variable name, if you can't understand it!

Reserved Words

These reserved words must not be used as variable names:

'False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield'

Note: Full list at the link below:

https://www.w3schools.com/python/python_ref_keywords.asp

Reserved Words and Errors

Syntax Error! (aka don't worry)

You don't have to memorise all those reserved words.

If you try and use them as a variable name,

Python will be sure to let you know its opinion on the matter.

```
>>> break = "me"
    break = "me"
        ^
SyntaxError: invalid syntax
```

Data Types

Python's data types are:

- the classification of data items
- they determine the operations performed on the data and;
- the structure in which the data is stored.

Variables can hold various types of data, and each type has its own unique capabilities.

Data Types

Some data types include:

- Text Type: str (string)
- Numeric Types: int (integer), float
- Sequence Types: list, tuple
- Mapping Type: dict (dictionary).

Full list: https://www.w3schools.com/python/python_datatypes.asp

Operators

Perform an action used to manipulate data.

Types of Operators:

- Assignment: assigns a value to a variable (=)
- Arithmetic (Mathematical): performs calculations (+, -, *, /, **)
- Comparison: Compares different values (< , >)
- Logical: Returns a true or false value. (True, False).

Operators

Full List here:

https://www.w3schools.com/python/python_operators.asp

Let's code!

Revision: IDE

IDLE has two windows.

1. IDLE File (where you code, text is editable)

mary-had-a-little-lamb.py - C:\Users\5000575\...

File Edit Format Run Options Window Help

```
# Firstname Lastname
# Student Number
# Mary Had a Little Lamb

# Print the rhyme

print("Mary had a little Lamb")
print()
print("Mary had a little Lamb,")
print("Little lamb, little lamb,")
print("Mary had a little lamb,")
print("Its fleece was white as snow.")
print()
print("And everywhere that Mary went,")
print("Mary went, Mary went,")
print("Everywhere that Mary went,")
print("The lamb was sure to go.")
```

Ln: 24 Col: 0

IDLE Shell 3.12.8

File Edit Shell Debug Options Window Help

```
Python 3.12.8 (tags/v3.12.8:2dc476b, Dec  3 2024, 19:30:04) [MSC v.1942
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\5000575\Desktop\mary-had-a-little-lamb.py
Mary had a little Lamb

Mary had a little Lamb,
Little lamb, little lamb,
Mary had a little lamb,
Its fleece was white as snow.
```

Ln: 16 Col: 0

IDLE Shell (output)

Try these in your Python Shell

1.

```
>>> x = 15
>>> x = x + 5
>>> print(x)
```

answer ?

2.

```
>>> x = 15
>>> x + 5 = x
>>> print(x)
```

answer ?

SyntaxError!!!

2.

```
>>> x = 15
>>> x + 5 = x
>>> print(x)
```

answer ?

```
x + 5 = x
```

SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?

Assignment Statements

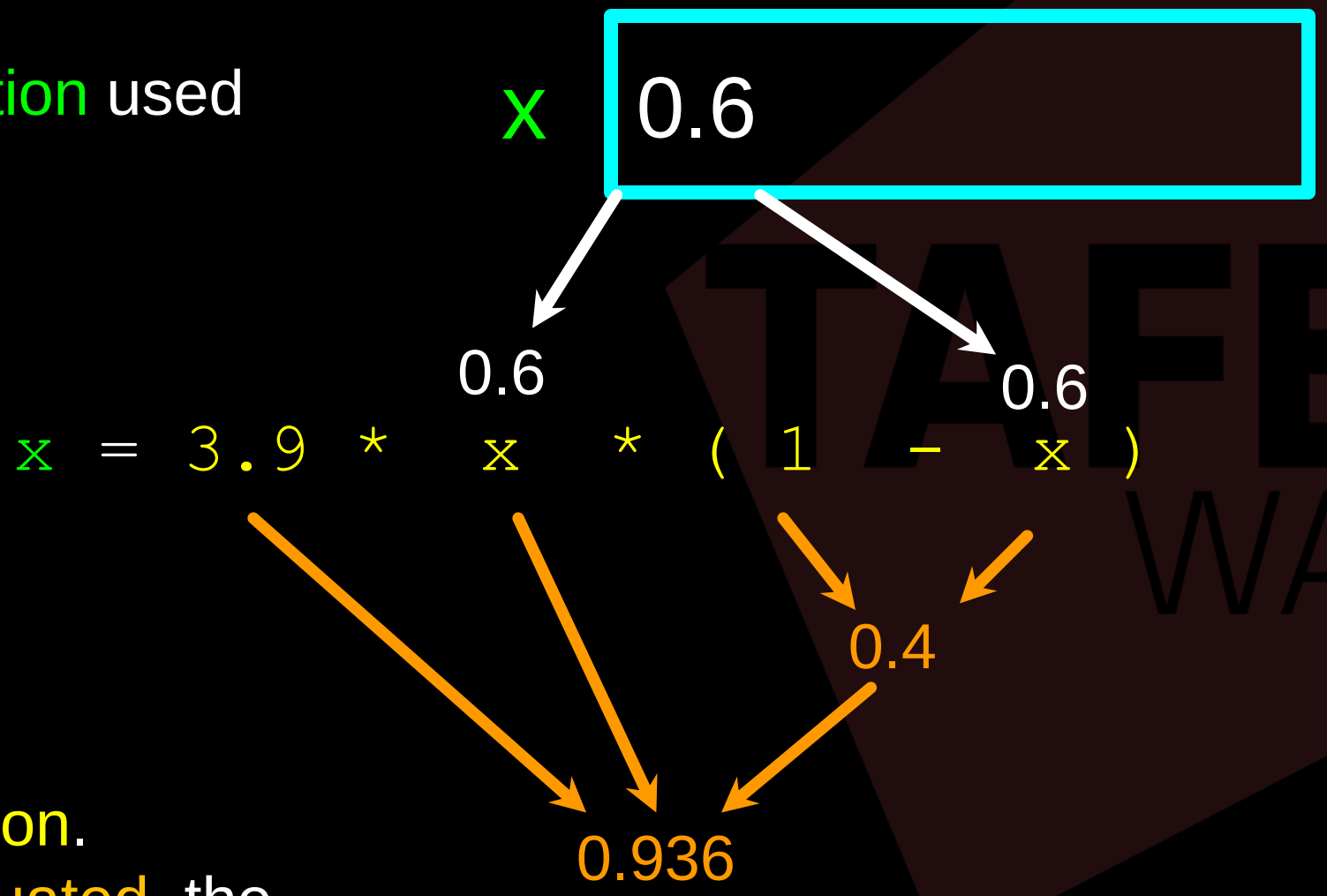
- An assignment operator (=) gives a value to a variable.
- The right-hand side of the = operator is evaluated first, and the result is stored in the variable on the left-hand side.

```
hours = 1
hours = hours + 1
print(hours)
```

Assignment statement
Assignment with expression
Print statement

variable assignment operator constant function

A variable is a memory location used to store a value (0.6)



The right side is an expression.
Once the expression is evaluated, the result is placed in (assigned to) x.

This part is more like ‘maths’

$$x = (x + 5) + 6 * x$$

Do you remember BIMDAS?

- Brackets
- Indices (exponents and roots)
- Multiplication
- Division
- Addition
- Subtraction

This acronym will help you remember the correct sequence (left to right) for solving mathematical expressions.

The old value (0.6) stored as the variable can be replaced with a new value (0.936).

$$x = \boxed{0.6}$$

0.6 0.6

```
x = 3.9 * x * (1-x)
x = 3.9 * 0.6 * 0.4
x = 0.936
```

expression

x ~~0.6~~ 0.936

Numeric Expressions

- Due to the lack of math symbols on computer keyboards - we use “computer-speak” to express math operations
- Asterisk (*) is multiplication
- Exponentiation (raise to a power) looks different than in math (**).

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Power
%	Remainder

Order of Evaluation

Python follows specific rules to determine the order of evaluation

This is called “operator precedence”

For example; in the expression 3 + 5 * 2, Python will perform the multiplication first (5 * 2 = 10), then 3 + 10, resulting in 13.

Order of Evaluation

Which operator "takes precedence" over the others?

```
x = 1 + 2 ** 3 / 4 * 5
```

```
>>> x = 1 + 2 ** 3 / 4 * 5
>>> print(x)
    11.0
```

Brackets ( )
Indices (**)
Multiplication (*)
Division (/)
Addition (+)
Subtraction (-)

Left to Right

```
1 + 2 ** 3 / 4 * 5
1 + 8 / 4 * 5
1 + 2 * 5
1 + 10
11
```

Constants

A **constant** in Python is a value that doesn't change.

It's like a fixed number or name that stays the same throughout your program.

For example, if you set `PI = 3.14`, you can use `PI` in your calculations, and it will always represent 3.14.

Constants

```
>>> print(123)
123
>>> print(98.6)
98.6
>>> print("Hello world")
Hello world
```

Let's try…

Type the following in the Python Shell.

```
>>>  "7" + "3"
>>>  "7" * 3
>>>  "7" + 3
>>>  7 + 3
>>>  33 / 3
```

Syntax Error!

```
>>>
>>> "7" + "3"
'73'
>>> "7" * 3
'777'
>>> "7" + 3
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    "7" + 3
TypeError: can only concatenate str (not "int") to str
>>> 7+3
10
>>> 33/3
11.0
>>>
```

What does "TypeError" Mean?

*Remember variables and constants have a type.

- A TypeError in Python occurs when an operation or function is applied to an object of an inappropriate type.
- Errors happen when you try to perform an operation on incompatible data types, like adding a string to an integer ("hello" + 5).

Examples of Types

- + means addition if something is a number and concatenate (join) if something is a string (" ")
- Integers are whole numbers: -14, -2, 0, 1, 100, 401233
- Floating Point Numbers are decimals: -2.5 , 0.0, 98.6
- Strings are characters, be careful... a number inside quotes " " is defined as a character.

String Conversions

To convert between integers and strings, you use int() and str().

- int(): Converts a string (text) or a float (decimal) to an integer (number).

```
number = int("123") # Converts the string "123" to the integer 123
```

- str(): Converts an integer or a float to a string.

```
text = str(123) # Converts the integer 123 to the string "123"
```

String Conversions

The float() function in Python is used to convert a number or a string that represents a number into a floating-point number (a number with a decimal point).

To convert a string or an integer to a float you use float().

```
decimal = float("12.34") # Converts the string "12.34" to the float 12.34
```

Type Conversion

Type	Comment
type()	are used to determine a variable's type
float(19)	
float('19')	
int ('19')	
str(19)	
ord("a")	will change the character "a" to its ascii code
chr(97)	will change the ascii code to a character

```
>>> days = 5
>>> type(days)
<class 'int'>
>>> float(19)
19.0
>>> float("19")
19.0
>>> int("19")
19
>>> ord("a")
97
>>> chr(97)
'a'
>>>
```

User Input

- We can instruct Python to pause and read data from the user using the input() function
- The input() function returns a string

```
>>> name = input('Who are you? ')
Who are you? Mary
>>> print('Welcome', name)
Welcome Mary
```

Simplified String Conversion Example

Instead of writing two lines of code, you can combine them into one line.

E.g.

```
number = input("How old are you? ")
number = int(number)
```

can also be typed as:

```
number = int(input("How old are you? "))
```

NOTE: An Error will occur if the string does not contain numeric characters.

More PEP8 Standards

Abbreviations are not English words. Do not shorten your variable names.

Constant variables: should be typed in UPPERCASE and use underscores:

e.g, DAYS_IN_A_WEEK = 7

Surround the following with a single whitespace:

- Assignment operators (= , += , -= , and so forth)
- Mathematical operators + - * ** % // usually we put spaces around lower precedence operators, but judgement is required.

More PEP8 Standards

Examples:

```
wickets_taken = wickets_taken + 1
gram = kilogram * 1000
circle_area = PI * radius ** 2
selling_price = (cost_price * profit) +
delivery_cost
```

Activities to complete

Practise what you have learnt today!

Locate your Week 2 Revision exercises folder and complete the following activities:

1. Mary Had a Little Lamb (personalised)
2. How to Make Toast.